

Cahier de cadrage — Application de Gestion des Congés

1. Contexte général

Dans le cadre du programme d'onboarding technique, il est demandé de développer une application complète de gestion des congés pour les fonctionnaires, permettant de digitaliser l'ensemble du cycle de vie des demandes : création, validation hiérarchique, signature, suivi statistique et traçabilité complète.

Ce projet a pour objectif de reproduire un contexte métier réel afin de permettre aux participants de mettre en pratique :

- le développement backend et frontend ;
 - les architectures modernes ;
 - les workflows métier ;
 - les bonnes pratiques de développement ;
 - la sécurité applicative ;
 - la qualité logicielle.
-

2. Objectifs pédagogiques

Le projet devra permettre de pratiquer :

Backend

- Java 17
- Spring Boot
- Spring Security
- JWT
- Spring Data JPA
- API REST
- DTO / Mapper
- gestion des exceptions
- logging
- audit

- validation métier
 - upload de fichiers
 - optimisation SQL
 - Liquibase
-

Frontend

- React
 - Redux Toolkit
 - React Router
 - Material UI
 - Atomic Design
 - gestion des formulaires
 - appels API
 - gestion d'état global
 - dashboards statistiques
-
-

3. Contexte métier

Le système permet :

- aux fonctionnaires de demander des congés ;
- aux responsables hiérarchiques de traiter les demandes ;
- aux signataires de signer les demandes validées ;
- à l'administration de suivre les statistiques et l'historique des demandes.

Chaque fonctionnaire appartient à :

- une direction ;
- une division;
- un service.

Le système doit également gérer :

- les intérim ;
- les quotas annuels ;
- les pièces justificatifs ;
- les jours fériés ;
- les weekends ;

- les workflows de validation.
-

4. Workflow métier

États possibles d'une demande

BROUILLON
SOUMISE
VISE_CHEF
REJETEE_CHEF
SIGNEE_DIRECTEUR
REJETEE_DIRECTEUR
ANNULEE

5. Règles métier

5.1 Création des demandes

- seul le propriétaire peut modifier sa demande tant qu'elle n'est pas validée par le chef hiérarchique ;
 - la saisie d'un intérim est obligatoire avant la soumission ;
 - les champs obligatoires doivent être validés ;
 - les dates doivent être cohérentes ;
 - Le nombre de jours doit être calculé automatiquement.
-

5.2 Validation

Validation chef hiérarchique

- Après validation, la demande devient non modifiable par le demandeur.

Validation directeur

- après validation, la demande devient non modifiable par :
 - le demandeur ;

- le chef hiérarchique.
-

5.3 Rejet

- un commentaire est obligatoire ;
-

5.4 Signature

- un document signé est obligatoire ;
 - seuls les signataires habilités peuvent signer ;
 - la demande passe à l'état "SIGNÉE".
-

5.5 Gestion des congés

Le système doit gérer :

- le quota annuel ;
- les jours fériés ;
- les weekends ;
- les chevauchements de périodes ;
- les congés maladie doivent être muni d'une pièce justificative ;
- les contraintes d'ancienneté.

Règles spécifiques :

- un intérim ne peut pas être en congé sur la même période ;
 - un fonctionnaire ne peut pas demander de congé avant un an d'ancienneté ;
 - Un fonctionnaire ne peut pas avoir plusieurs congés qui se chevauchent.
-

6. Gestion des rôles et habilitations

Rôles applicatifs

Rôle	Description
ADMIN	administration complète
FONCTIONNAIRE	création et suivi des demandes
CHEF_HIERARCHIE	validation des demandes
SIGNATAIRE	signature des demandes

7. Fonctionnalités attendues

7.1 Authentification & Sécurité

Fonctionnalités

- authentification JWT ;
 - gestion des rôles ;
 - refresh token ;
 - expiration des tokens ;
 - limitation des tentatives de connexion ;
 - logout sécurisé.
-

Sécurité

- endpoints sécurisés ;
 - contrôle des accès par rôle ;
 - gestion des permissions ;
 - protection des ressources.
-

7.2 Logging & Audit

Le système doit journaliser :

- les connexions ;
- les erreurs ;

- les validations ;
- les rejets ;
- les signatures ;
- les modifications.

Les logs doivent contenir :

- utilisateur ;
- date ;
- action ;
- durée ;

Les informations sensibles ne doivent jamais être loggées :

- mot de passe ;
 - JWT ;
 - secrets.
-

7.3 Gestion des fonctionnaires

Fonctionnalités :

- CRUD ;
- recherche multicritère ;
- pagination ;
- export Excel/PDF ;
- activation/désactivation.

Champs obligatoires :

- nom ;
 - prénom ;
 - PPR ;
 - date début fonction ;
 - grade.
 - service d'attachement
-

7.4 Gestion des demandes

Fonctionnalités :

- création ;

- modification ;
- soumission ;
- annulation ;
- consultation ;
- historique des états ;
- suivi workflow.

Champs obligatoires :

- date départ ;
 - date arrivée ;
 - durée ;
 - date demande ;
 - type congé ;
 - année administrative.
-

7.5 Gestion documentaire

Fonctionnalités :

- upload PDF/image ;
 - téléchargement ;
 - suppression ;
 - prévisualisation ;
 - validation extension ;
 - validation taille fichier.
-

7.6 Dashboard & Reporting

Le système doit afficher :

- nombre de demandes par état ;
 - demandes par direction ;
 - demandes par type de congé ;
 - demandes en cours de traitement ;
 - taux validation/rejet ;
 - statistiques mensuelles.
-

7.7 Notifications email

Notifications automatiques :

- soumission demande ;
 - validation ;
 - rejet ;
 - signature ;
 - rappel traitement.
-

8. Contraintes techniques

Backend

Architecture recommandée :

controller
service
repository
dto
mapper
entity
security
config
exception

Frontend

Architecture recommandée :

pages
components
atoms
molecules

9. Bonnes pratiques attendues

Backend

- méthodes courtes ;
 - séparation des responsabilités ;
 - validation centralisée(validation des champs par les annotations);
 - DTO obligatoires ;
 - gestion globale des exceptions(@RestControllerAdvice) ;
 - logs structurés.
-

Frontend

- composants réutilisables ;
 - Atomic Design ;
 - centralisation appels API ;
 - séparation logique/UI ;
 - gestion loading/error states.
-

10. Critères qualité

Le projet sera évalué sur :

- qualité du code ;
 - architecture ;
 - sécurité ;
 - maintenabilité ;
 - performance ;
 - UX/UI ;
 - respect des bonnes pratiques ;
-

11. Livrables attendus

Backend

- projet Spring Boot fonctionnel ;
 - scripts Liquibase ;
 - documentation Swagger.
-

Frontend

- application React fonctionnelle et responsive.

12. pré-requis de l'environnement de travail:

- docker
- docker compose pour ajouter le container de sql server
- créer la base de donnée et l'utilisateur d'accès à la base de donnée
- maven

Frontend

- Node.js LTS
- Yarn
- utiliser l'extension sql server en vscode
- Installer ESLint et StyleLint
- utiliser prettier pour le formatage

database

```
CREATE DATABASE GestionCongeAppDb_Loc;
CREATE LOGIN GestionCongeAppDbUser_Loc WITH PASSWORD = 'LocPassword123',
DEFAULT_DATABASE=GestionCongeAppDb_Loc;
USE GestionCongeAppDb_Loc;
CREATE USER GestionCongeAppDbUser_Loc FOR LOGIN GestionCongeAppDbUser_Loc;
GRANT CONTROL TO GestionCongeAppDbUser_Loc;
```